

3. Hafta

JavaScript'ten TypeScript'e: Dil Modeli, Araçlar ve Asenkron Programlama

© Öğrenme Hedefleri

Bu bölümü tamamladığınızda:

- JavaScript'in dil modelini (değer/referans semantiği, `let/const`, fonksiyon tanımları ve ok fonksiyonu, closure, nesne/dizi, prototip zinciri ve `class` sözdizimi) C++'taki karşılıklarıyla kıyaslayarak açıklayıp tarayıcı konsolunda deneyebileceksiniz;
- ES modüllerini (`import/export`), npm'in paket/ bağımlılık yönetimini (`package.json`, `npm install`, `scripts`) ve Vite ile bir TypeScript projesini kurup geliştirme sunucusuyla (`npm run dev`) ve derleme çıktısıyla (`npm run build`) çalıştırabileceksiniz;
- TypeScript'in temel tiplerini, `interface/type`, union ve literal tipleri, `readonly` ve generic (`Liste<T>`) yapıları kullanarak Etkinlik/Bilet Sisteminin Etkinlik, Koltuk, Bilet alan modeli tiplerini tanımlayıp tip daraltma yapabilecek ve `tsc --noEmit` ile tip güvenliğini doğrulayabileceksiniz;
- Olay döngüsünü, Promise'i ve `async/await`'i, `setTimeout`'u ve temel DOM API'sini (`querySelector`, `addEventListener`, öge oluşturma) kullanarak bir veri dizisinden dinamik bir liste üretebilecek ve kullanıcı etkileşimine (tıklama, form gönderimi) tepki verebileceksiniz;
- Bu becerileri birleştirip Hafta 2'nin statik etkinlik sayfasını TypeScript ile canlandırarak (Etkinlik/Koltuk/Bilet tiplerinden dizi üretme, koltuk seçimi, seçili koltuk sayısı/tutar hesaplama, form gönderiminde `preventDefault` + konsola özet) Etkinlik/ Bilet Sistemine ilk davranış katmanını ekleyebileceksiniz.

Hafta 2'de yazdığınız statik etkinlik sayfası, koltuk planı ve bilet formu **hiçbir şey yapmıyordu** — koltuğa tıklayınca renk değişmiyor, form gönderilince sayfa boşuna yenileniyordu. Bu hafta o HTML/CSS yapısına dokunmadan, JavaScript'in dil modelini öğrenip TypeScript'e geçerek, DOM üzerinden ilk kez **davranış** ekleyeceksiniz. Dört ünite boyunca sırasıyla JavaScript'in kendisini (dil modeli: değer/referans, fonksiyonlar, closure, prototip), o dili paketleyen araçları (ES modülleri, npm, Vite), üstüne eklenen tip katmanını (TypeScript: interface, union, generic)

ve son olarak bu üçünü birleştiren asenkron/DOM programlamayı göreceksiniz. Bu hafta React **henüz yoktur** — tüm DOM işlemleri düz TypeScript ve tarayıcının kendi API'siyle elle yazılacak; Ünite 4'ün sonunda, bunu elle yapmanın neden hızla karmaşıklaştığını göreceksiniz — bu da Hafta 4'ün React'e neden ihtiyaç duyduğunuzu gösterecektir. Öğrendiğiniz her yeni JS/TS kavramı, YG I/II'de gördüğünüz bir C/C++ kavramıyla köprülenerek anlatılacaktır: değer/referans semantiği C++'ın kopya/pointer ayrımıyla, generic YG I'in `Liste<T>` şablon sınıfıyla, tek iş parçacıklı olay döngüsü YG II'nin `std::thread` ile karşılaştırılacaktır.

3.1 JavaScript Dil Modeli: Değer/Referans, Fonksiyonlar, Closure, Nesne ve Prototip

1 • Ön-öğretim Neden Önemli?

Bir fonksiyona bir diziyi parametre olarak verip fonksiyon içinde dizinin bir elemanını değiştirdiğinizde, fonksiyon bittikten sonra dışarıdaki orijinal dizi de değişmiş olur — ama aynısını bir sayıyla yaparsanız dışarıdaki sayı değişmez. Deneyimsiz bir gözle bu bir “hata” gibi görünür; aslında JavaScript'in değer/referans ayrımının doğal, öngörülebilir bir sonucudur. Bu ünite bu ayrımı netleştirir.

1 • Ön-öğretim Ön Bilgi

YG I/II'de C/C++'ta öğrendiğiniz değer/referans/pointer ayrımını (`int b = a;` bir kopyadır; `T&` pointer bir referanstır) ve STL container'larını bir fonksiyona değere göre mi referansa göre mi geçirdiğiniz kararını hatırlayın. Bu ünite aynı ayrımı JavaScript'te yeniden kurar — ama JS'te seçim hakkınız yoktur: nesne ve dizi her zaman referanstır.

Hafta 2'de yazdığınız `index.html` ve `etkinlik-detay.html` sayfaları tamamen statikti; hiçbir JavaScript kodu içermiyorlardı. Bu hafta bu sayfalara davranış ekleyeceğimiz dilin — JavaScript'in, sonra da onun tip sistemiyle güçlendirilmiş hâli TypeScript'in — temel dil modelini kuruyorsunuz. Bu ünite, C/C++/Python bildiğiniz varsayımıyla, her yeni JS kavramını size zaten tanıdık gelen bir karşılıkla köprüleyerek anlatır.

3.1.1 `let/const` ve blok kapsamı

JavaScript'te bir değişken üç anahtar kelimeden biriyle tanımlanır: `var`, `let`, `const`. Bu derste yalnızca sonuncu ikisini kullanacaksınız. `let` ile tanımlanan bir değişkenin değeri sonradan değiştirilebilir; `const` ile tanımlanan bir değişkenin **bağı** (değişkenin hangi değere işaret ettiği) bir daha değiştirilemez — bu, C++'taki `const` ile aynı fikirdir, ama önemli bir ayrıntıyla: `const` bir nesne ya da diziye bağlıysa, nesnenin/dizinin **içeriği** yine değiştirilebilir; yalnızca değişkenin başka bir nesneye/diziye **yeniden atanması** engellenir. `let` ve `const`, tanımlandıkları `{ }` bloğunun (bir `if`, `for` ya da fonksiyon gövdesinin) dışında görünmez — buna **blok kapsamı** denir ve C++'taki `{ }` blok kapsamıyla birebir aynı davranır.

Eski `var` anahtar kelimesi ise **fonksiyon kapsamlıdır**: bir `if` bloğu içinde `var` ile tanımlanan bir değişken, o bloğun dışında da görünür kalır — bu, öngörülemez

blok kapsamı:
bir değişkenin yalnızca tanımlandığı süslü parantez bloğu içinde görünür/erişilebilir olması (*block scope*).

hatalara yol açan, modern JavaScript'te terk edilmiş bir davranıştır. Bu ders boyunca `var` kullanılmayacak; her yerde `let` ya da `const` tercih edilecektir.

3.1.2 Değer semantiği: ilkel tipler kopyalanarak atanır

JavaScript'in beş temel **ilkel** (*primitive*) tipi vardır: `number`, `string`, `boolean`, `undefined`, `null`. Bir ilkel değeri bir değişkene atadığınızda ya da başka bir değişkene kopyaladığınızda, JavaScript **değer semantiği**ni izler: iki değişken artık birbirinden tamamen bağımsız iki ayrı kopyadır, biri değişse diğeri **etkilenmez**. Aşağıdaki kesitin ilk bölümü bunu gösterir: `b = a` satırından sonra `b`'yi `10`'a değiştirmek, `a` üzerinde hiçbir etki bırakmaz — tıpkı C++'ta `int b = a;` yazdığınızda `b`'nin `a`'dan bağımsız yeni bir bellek hücresi olması gibi.

değer semantiği: bir değişkenin başka bir değişkene atanırken değerinin bağımsız bir kopyasının oluşturulması (*value semantics*).

Analoji

Değer semantiğini bir sayıyı kağıda yazıp fotokopisini vermeye benzetin: fotokopi kağıdını (değeri) sonradan değiştirmek, elinizdeki orijinal kağıdı hiç etkilemez — ikisi artık birbirinden bağımsız iki ayrı kağıttır.

3.1.3 Referans semantiği: nesne ve dizi paylaşılan bir belleğe işaret eder

Nesne ve dizi değişkenleri ise tamamen farklı davranır. Bir nesneyi ya da diziyi başka bir değişkene atadığınızda JavaScript **referans semantiği**ni izler: değişken kopyalanan bir değer taşımaz, ortak bir bellek alanına işaret eden bir bağlantı taşır. İki değişken aynı nesneyi/diziyi işaret edebilir; birinin üzerinden yapılan bir değişiklik — ör. `push` ile bir eleman ekleme — diğer değişkenden de **görünür**, çünkü aslında ortada tek bir dizi/nesne vardır, yalnız ona iki isimle erişilmektedir. Aşağıdaki kesitin ikinci ve üçüncü bölümleri bu farkı, önce iki değişken arasında, sonra da bir fonksiyona parametre geçirirken gösterir:

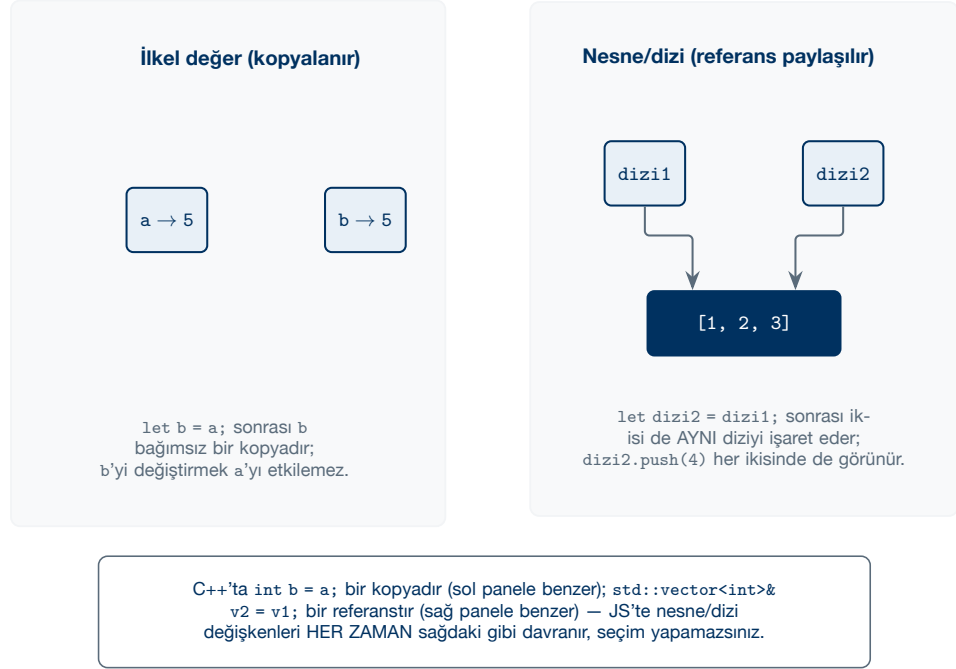
```
1 let a = 5;
2 let b = a;
3 b = 10;
4 console.log("a:", a);
5
6 let dizil = [1, 2, 3];
7 let dizi2 = dizil;
8 dizi2.push(4);
9 console.log("dizil:", dizil);
10
11 function ekle(dizi) {
12   dizi.push(99);
13 }
14 let sayilar = [1, 2, 3];
15 ekle(sayilar);
16 console.log("sayilar:", sayilar);
```

Kesitin son bölümünde `ekle(sayilar)` çağrısı, `ekle` fonksiyonunun içindeki `dizi` parametresine `sayilar` dizisinin kendisini (bir kopyasını değil) bağlar; `dizi.push(99)` bu ortak diziyi mutasyona uğratar ve fonksiyon bittikten sonra dışarıdaki `sayilar` değişkeninden de `99` görünür hâle gelir.

Analoji

Referans semantiğini bir evin anahtarını iki kişiye vermeye benzetin: aynı evin anahtarını (referansı) iki kişiye verirseniz, biri evdeki bir eşyayı taşırsa diğeri de bu değişikliği — eve girdiğinde — görür. JS'te nesne/dizi değişkenleri her zaman anahtar gibi davranır; kimse “kendi fotokopisini” almaz.

Şekil 3.1 bu iki davranışı yan yana gösterir: solda birbirinden bağımsız iki kutu (kopyalama), sağda iki değişkenin ortak tek bir kutuya birlikte işaret etmesi (paylaşılan referans).



Şekil 3.1: İlkel değerlerin kopyalanması (sol panel) ile nesne/dizi değerlerinin ortak bir belleğe referansla paylaşılması (sağ panel); C++'ın `int b = a;` (kopya) ile T&/pointer (referans) ayrımıyla karşılaştırma.

Ortak Hata / Tuzak

C++ deneyiminden gelen bir öğrenci sıklıkla “JavaScript'te bir diziyi fonksiyona verirken kopyalanır mı, referansla mı geçer, ben mi seçerim” diye sorar. Cevap: **seçim hakkınız yoktur**. C++'ta bir `std::vector<int>` parametresini değere göre (parametre tipi `std::vector<int>`) ya da referansa göre (parametre tipi `std::vector<int>&`) geçirmeyi siz seçersiniz; JavaScript'te nesne ve dizi parametreleri her zaman referans olarak geçer, bunun “değere göre” bir eşdeğeri yoktur. Bir diziyi değiştirmeden bırakmak istiyorsanız, önce (spread operatörüyle, birazdan göreceksiniz) bir kopyasını oluşturmanız gerekir.

3.1.4 Fonksiyonlar: bildirim, ifade ve ok fonksiyonu

JavaScript'te bir fonksiyon iki temel biçimde tanımlanabilir. Birincisi **fonksiyon bildirimidir** (*function declaration*) — bu, C++'taki sıradan bir fonksiyon tanımına ben-

zer ve programın **her yerinden** çağrılabilir (JavaScript motoru bu tanımlı dosyanın en başına “taşır”, buna *hoisting* denir):

```
1 function carp(x, y) { return x * y; }
```

İkincisi **fonksiyon ifadesidir** (*function expression*) — burada fonksiyon, sıradan bir değer gibi bir değişkene atanır ve yalnızca bu atama satırından sonra çağrılabilir:

```
1 const topla = function (x, y) { return x + y; };
```

```
1 function carp(x, y) {
2   return x * y;
3 }
4
5 const topla = function (x, y) {
6   return x + y;
7 };
8
9 console.log("carp(3, 4):", carp(3, 4));
10 console.log("topla(3, 4):", topla(3, 4));
```

ok fonksiyonu:
=> ile yazılan, kısa söz dizimine ve leksik *this* bağlamına sahip fonksiyon (*arrow function*).

Üçüncü ve bu derste en sık kullanacağınız biçim **ok fonksiyonudur**: $(x) \Rightarrow x * 2$ biçiminde, tek satırlık kısa bir söz dizimiyle yazılır. Ok fonksiyonu yalnızca kısalık için değil, *this*’in davranışı yüzünden de tercih edilir: bir normal fonksiyonun içindeki *this*, fonksiyonun **çağrıldığı yere** göre belirlenir (bir nesnenin metodu olarak çağrılırsa *this* o nesnedir); bir ok fonksiyonunun içindeki *this* ise **tanımlandığı** (leksik) kapsamdan gelir, hiçbir zaman çağırılma biçimine göre değişmez. Aşağıdaki kesit bu farkı doğrudan gösterir:

```
1 const sayac = {
2   deger: 0,
3   artirNormal: function () {
4     this.deger++;
5     console.log("artirNormal sonrasi deger:", this.deger);
6   },
7   artirOk: () => {
8     console.log("artirOk içindeki this sayac mı?", this === sayac);
9   },
10 };
11
12 sayac.artirNormal();
13 sayac.artirOk();
```

`sayac.artirNormal()` çağrısında *this*, `sayac` nesnesinin kendisidir — çünkü fonksiyon `sayac`. üzerinden, bir nesne metodu olarak çağırılmıştır. `sayac.artirOk()` çağrısında ise *this*, `sayac` değildir — ok fonksiyonu tanımlandığı yerdeki (bu dosyanın en dış kapsamındaki) *this*’i kullanır, `sayac`. üzerinden çağırılmış olması hiçbir şey değiştirmez.

💡 İpucu

Pratik kural: bir nesnenin metodunu tanımlarken (o metodun kendi *this*’ine ihtiyacı varsa) normal fonksiyon söz dizimini kullanın; kısa, bağımsız yardımcı fonksiyonlarda (ör. `map/filter` içine geçirilen küçük fonksiyonlarda, birazdan göreceksiniz) ok fonksiyonunu tercih edin.

3.1.5 Closure: kapanan kapsama erişmeye devam etmek

closure: bir fonksiyonun, tanımlandığı kapsam kapansa bile o kapsamdaki değişkenlere erişmeye devam etmesi (*closure*).

Bir iç fonksiyon, tanımlandığı dış fonksiyon çoktan çalışmasını bitirmiş olsa bile, o dış fonksiyonun değişkenlerine erişmeye devam edebilir. Buna **closure** denir. Bu, C++'ta doğrudan bir karşılığı olmayan, çalışma zamanına özgü bir JavaScript mekanizmasıdır — en yakın C++ benzetmesi, bir değişkeni **[&]** ile **referansla** yakalayan bir lambda ifadesidir, ama JS'teki closure ömrü boyunca (fonksiyon çağrıldığı sürece) geçerli kalır.

Analoji

Closure'ı bir dolabın anahtarını yalnızca ona ihtiyacı olan görevliye vermeye benzetin: iç fonksiyon, oluşturulduğu “oda”daki (dış fonksiyonun değişkenleri) dolabın anahtarını elinde tutar; oda görünürde kapansa bile (dış fonksiyon bitse bile) o dolaba erişmeye devam eder.

Aşağıdaki `olusturSayac` fonksiyonu her çağrıldığında kendi bağımsız `deger` değişkenine sahip yeni bir fonksiyon döndürür:

```
1 function olusturSayac() {
2   let deger = 0;
3   return function () {
4     deger++;
5     return deger;
6   };
7 }
8
9 const sayac1 = olusturSayac();
10 const sayac2 = olusturSayac();
11
12 console.log("sayac1():", sayac1());
13 console.log("sayac1():", sayac1());
14 console.log("sayac2():", sayac2());
15 console.log("sayac1():", sayac1());
```

Örnek Olay 3.1: Bağımsız sayaçlar oluşturan bir closure

Adım adım izleyin: `olusturSayac()` ilk çağrıldığında, içeride `deger = 0` olan bir kapsam oluşur ve bu kapsamı “hatırlayan” bir iç fonksiyon döndürülür — bu döndürülen fonksiyon `sayac1`'e atanır. `olusturSayac()` İkinci kez çağrıldığında, tamamen yeni ve bağımsız bir `deger` kapsamı oluşur; bu ikinci fonksiyon `sayac2`'ye atanır. `sayac1()` her çağrıldığında yalnızca kendi `deger`'ini artırır (1, 2, 3, ...); `sayac2()` ise kendi bağımsız `deger`'ini sıfırdan başlatıp artırır — iki sayaç birbirinin haberi olmadan, kendi kapanmış kapsamlarında yaşarlar. Bu yüzden çıktıda `sayac1()` üçüncü kez çağrıldığında 3 görürsünüz, ama `sayac2()` hâlâ kendi ilk çağrısında 1 döndürmüştür.

3.1.6 Nesne, dizi ve yaygın dizi metotları

Bir **nesne literal** (*object literal*), süslü parantez içinde **alan**: **değer** çiftleriyle yazılır — C++'taki bir **struct** örneğine benzer, ama JavaScript'te tip önceden tanımlanmaz, alanlar doğrudan literal içinde belirlenir:

```
1 const etkinlik = {
2   ad: "Konser",
```

```

3   tarih: "2026-11-14",
4   kapasite: 500,
5 };
6 console.log("etkinlik:", etkinlik);

```

Bir **dizi literal** (*array literal*) ise köşeli parantez içinde elemanların virgülle ayrılmasıyla yazılır, ör. `[1, 2, 3]`. Diziler üzerinde en sık kullanacağınız üç metot şunlardır: **map** her elemanı bir dönüşüm fonksiyonundan geçirip aynı uzunlukta yeni bir dizi üretir; **filter** yalnızca bir koşulu sağlayan elemanlardan yeni (muhtemelen daha kısa) bir dizi üretir; **find** koşulu sağlayan ilk elemanı döndürür, hiçbiri sağlamıyorsa **undefined** döner. Bu üç metot da orijinal diziyi **değiştirmez**, her zaman yeni bir sonuç üretir — C++’taki `std::transform/std::copy_if/std::find_if` ile aynı fikri taşırlar, ama STL’in aksine ayrı bir hedef container önceden ayırmanız gerekmez.

```

1  const etkinlikler = [
2    { ad: "Bahar Konseri", kapasite: 500 },
3    { ad: "Kariyer Günleri", kapasite: 800 },
4    { ad: "Tiyatro Gecesi", kapasite: 250 },
5  ];
6
7  const adlar = etkinlikler.map((e) => e.ad);
8  console.log("adlar:", adlar);
9
10 const buyukEtkinlikler = etkinlikler.filter((e) => e.kapasite >= 500);
11 console.log("buyukEtkinlikler:", buyukEtkinlikler);
12
13 const tiyatro = etkinlikler.find((e) => e.ad === "Tiyatro Gecesi");
14 console.log("tiyatro:", tiyatro);

```

Bu üç metot, Hf.4’ten itibaren Etkinlik/Bilet Sisteminin `Etkinlik[]` ve `Koltuk[]` dizileri üzerinde sürekli kullanacağınız temel araçlardır — ör. yalnızca “boş” koltukları **filter**’lamak ya da bir etkinlik dizisinden yalnızca başlıkları **map**’lemek.

Bir dizinin/nesnenin sığ (*shallow*) bir kopyasını oluşturmak için **... yayma (spread) operatörü** kullanılır: `[...dizi]` ya da `{...nesne}`, orijinal referansı değil, yeni bir dizi/nesne referansı üretir. Bu, biraz önce gördüğünüz “referans her zaman paylaşılır” kuralının istisnası değil, ondan **kaçmanın** yoludur: bir diziyi değiştirmeden bir kopyasını almak istediğinizde spread operatörünü kullanırsınız.

```

1  const etkinliklerKopya = [...etkinlikler];
2  etkinliklerKopya.push({ ad: "Yeni Etkinlik", kapasite: 100 });
3  console.log("etkinlikler.length:", etkinlikler.length);
4  console.log("etkinliklerKopya.length:", etkinliklerKopya.length);
5
6  const etkinlikGuncel = { ...etkinlik, kapasite: 600 };
7  console.log("etkinlik.kapasite:", etkinlik.kapasite);
8  console.log("etkinlikGuncel.kapasite:", etkinlikGuncel.kapasite);

```

3.1.7 Prototip zinciri ve class sözdizimi

JavaScript’te her nesnenin bir **prototipi** vardır ve bir nesnede bulunmayan bir özelliğe erişmeye çalıştığınızda, JS motoru bu özelliği önce nesnenin kendisinde arar, bulamazsa **prototip zincirinde** yukarı doğru aramaya devam eder.

prototip zinciri:
bir nesnede
bulunmayan özelliğin,
nesnenin prototipinde
ve onun prototipinde
aranmasını sağlayan
zincir (*prototype chain*).

Analoji

Prototip zincirini bir ailede soyadı sorusuna benzetin: bir kişiye soyadını sorduğunuzda kendisinde yoksa (yeni evlenmiş olabilir), önce ebeveynine, o da yoksa büyük ebeveynine bakılır. JS de bir nesnede bulunmayan özelliği önce onun prototipinde, sonra onun prototipinde arar.

```

1 const katilimciProto = {
2   selamla() {
3     return `Merhaba, ben ${this.ad}`;
4   },
5 };
6
7 const katilimcil = Object.create(katilimciProto);
8 katilimcil.ad = "Ayşe";
9
10 console.log(katilimcil.selamla());
11 console.log("kendi özelliği mi?", Object.hasOwn(katilimcil, "selamla"));

```

Yukarıdaki kesitte `katilimcil`'in kendi üzerinde `selamla` adlı bir özellik yoktur; `Object.create(katilimciProto)` onun prototipini `katilimciProto` olarak bağlamıştır, bu yüzden `katilimcil.selamla()` çağrısı, bu metodu prototipte bulup çalıştırır. `Object.hasOwn(katilimcil, "selamla")` bunun `false` olduğunu, yani `selamla`'nın `katilimcil`'in **kendi** özelliği olmadığını doğrular.

Modern JavaScript'te bu mekanizmayı elle `Object.create` ile kurmak yerine, aynı davranışı çok daha okunaklı bir söz diziminde — ES6 **class** söz dizimiyle — yazabilirsiniz. `class`, C++'taki `class` anahtar kelimesine görünüşte çok benzer, ama JavaScript'te `class`'ın kendisi yeni bir mekanizma değildir — prototip zincirinin üstüne kurulmuş, okunaklı bir kısayoldur ("syntactic sugar"). Aşağıdaki kesit tam olarak aynı davranışı `class` ile yazar:

```

1 class Katilimci {
2   constructor(ad) {
3     this.ad = ad;
4   }
5
6   selamla() {
7     return `Merhaba, ben ${this.ad}`;
8   }
9 }
10
11 const katilimci2 = new Katilimci("Mehmet");
12 console.log(katilimci2.selamla());
13 console.log(
14   "selamla prototipte mi?",
15   Object.hasOwn(Katilimci.prototype, "selamla")
16 );

```

Son satır (`Object.hasOwn(Katilimci.prototype, "selamla")` — `true`) bunu kanıtlar: `class` söz dizimiyle tanımlanan `selamla` metodu da, tıpkı elle kurduğunuz örnekte olduğu gibi, aslında `Katilimci.prototype` üzerinde durur — `class` size yalnızca daha tanıdık bir görünüm sunar, JS motorunun altındaki mekanizmayı değiştirmez.

 İpucu

Referansı kalmayan bir nesne (artık hiçbir değişken ona işaret etmiyorsa) JavaScript motoru tarafından **otomatik** olarak temizlenir — buna çöp toplama (*garbage collection*, GC) denir. Bu, C++'ın elle **delete** çağırmanızı ya da bir RAIL yıkıcı (*destructor*) yazmanızı gerektiren bellek yönetiminden temelden farklıdır: JS'te bellek serbest bırakmayı siz değil, motor yönetir — avantajı daha az hata riski, dezavantajı ise serbest bırakılma anının tam olarak ne zaman gerçekleşeceği üzerinde kontrolünüzün olmamasıdır.

3.1.8 Dinamik tiplendirme: TypeScript'in neden var olduğunun ilk ipucu

Buraya kadar gördüğünüz her örnekte bir değişkenin tipini hiç yazmadınız — `let a = 5;` yazdınız, `int` ya da `number` belirtmediniz. Bunun nedeni JavaScript'in **dinamik tiplendirme** kullanmasıdır: aynı `a` değişkenine önce bir sayı, sonra bir metin, sonra bir nesne atayabilirsiniz — derleyici (böyle bir kavram JS'te zaten yoktur) buna itiraz etmez. C++'ın derleme-zamanı statik tiplendirme bunun tam tersidir: bir `int` değişkenine bir `std::string` atamaya çalıştığınızda kod hiç derlenmez. JavaScript'in bu esnekliği hızlı prototiplendirme için avantajlıdır, ama büyüyen bir projede bir fonksiyona yanlış tipte bir değer geçirmek gibi hatalar kod çalışana kadar fark edilmeyebilir. Ünite 3, bu esnekliğin üstüne — onu ortadan kaldırmadan — isteğe bağlı bir güvenlik katmanı ekleyen TypeScript'i tam olarak bu sorunu çözmek için tanıttacaktır.

dinamik tiplendirme: bir değişkenin türünün çalışma zamanında değişebilmesi (*dynamic typing*).

3.1.9 Tarayıcı konsolunda deneme

Bu ünitenin tüm örnekleri iki ortamda da çalıştırılabilir: terminalde `node dosya.js` ile (Node.js kurulu olduğu sürece, ekstra kurulum gerekmez) ya da bir tarayıcının Geliştirici Araçları'ndaki **Konsol** (*Console*) sekmesinde, ifadeleri tek tek yapıştırıp Enter'a basarak. Konsol, her ifadenin sonucunu ve `console.log` çağrılarının çıktısını satır satır gösterir — bu, kodu tekrar tekrar derleyip çalıştırmadan hızlı deneme yapmanın en pratik yoludur.

 EKRAN GÖRÜNTÜSÜ YER TUTUCU

Tarayıcı Geliştirici Araçları'nın Konsol sekmesinde `deger-referans.js` içeriğinin satır satır çalıştırılması ve `console.log` çıktılarının (`a: 5`, `dizi1: [1, 2, 3, 4]`, `sayılar: [1, 2, 3, 99]`) görünmesi.

[assets/ss/h03-devtools-console-deger-referans.png](#)

3 • Katılım Şimdi Siz Deneyin

Aşağıdaki kodu çalıştırmadan önce her `console.log`'un ne basacağını tahmin edin, sonra tarayıcı konsolunda ya da `node deger-referans.js` ile deneyin:

```
let a = 5;
let b = a;
b = 10;
console.log(a); // ?

let dizi1 = [1, 2, 3];
let dizi2 = dizi1;
```

```
dizi2.push(4);
console.log(dizi1); // ?
```

Geribildirim. *a* hâlâ 5'tir — *number* bir değer tipidir, *b = a* bir kopya oluşturur, *b*'yi sonradan değiştirmek *a*'yı etkilemez. *dizi1* artık [1, 2, 3, 4]'tür — *dizi2 = dizi1* aynı diziye işaret eden ikinci bir referans oluşturur; *push* ile yapılan değişiklik, her iki değişkenden de görünen tek diziyi etkiler.

Kendinizi Sınavın 3.1

Aşağıdaki kodda `console.log(sayilar)` çıktısı ne olur ve neden?

```
function ekle(dizi) { dizi.push(99); }
let sayilar = [1, 2, 3];
ekle(sayilar);
console.log(sayilar);
```

5 • İzleme Aktarım — Başka Nerede?

Bu haftanın `Etkinlik[]` ve `Koltuk[]` dizileri de birer nesne dizisidir; Hf.4'te React'in `state` güncellemelerinde “mevcut diziyi elle değiştirmek” (mutasyon) ile “yeni bir dizi oluşturmak” arasındaki kritik fark, tam olarak bu ünitenin öğrettiği referans semantiğine dayanacaktır — React bir mutasyonu fark edemez, yalnızca yeni bir referans gördüğünde ekranı yeniden çizer.

3.2 Modüller ve Araç Zinciri: ES Modülleri, npm, Vite

1 • Ön-öğretim Neden Önemli?

Hafta 2'de yazdığınız `index.html` dosyasını tarayıcıda çift tıklayarak açabiliyordunuz — hiçbir kurulum gerekmiyordu. Ama TypeScript kodu tarayıcının doğrudan anlamadığı bir dildir; önce JavaScript'e çevrilmesi (derlenmesi) gerekir. Peki bu çeviriyi kim, ne zaman, nasıl yapacak? Bu ünite bu boşluğu doldurur.

1 • Ön-öğretim Ön Bilgi

Hafta 1 Ünite 2'nin “web sunucusu” kavramını ve Ünite 3'ün “SPA” (*Single Page Application*) kavramını hatırlayın. Hafta 2'nin salt `.html/.css` dosyaları hiçbir derleme aracı gerektirmiyordu — hiç `package.json` dosyanız yoktu. Bu ünite ilk kez bir araç zinciri kuracaksınız.

Ünite 1'de JavaScript'in dilini öğrendiniz; ama TypeScript kodu, düzenli bir `.ts` uzantılı dosyaya yazdığınız halde tarayıcı tarafından **doğrudan** çalıştırılmaz. Tarayıcılar yalnızca JavaScript anlar; TypeScript'in tarayıcıya ulaşabilmesi için önce JavaScript'e **çevrilmesi** (derlenmesi) gerekir. Bu ünite, bu çeviriyi kimin yaptığını, projenizin parçalarını nasıl birbirine bağladığınızı ve bu haftadan itibaren `kod_ornekleri/hafta03/istemci/` klasöründe duran gerçek projenizin nasıl kurulup çalıştırıldığını anlatır.

3.2.1 ES modülleri: her dosya kendi kapsamını korur

Büyüyen bir projede tüm kodu tek bir dosyaya yazmak sürdürülemez; kodu dosyalara bölmeniz gerekir. Ama bir dosyayı diğerine **bağlamanın** güvenli bir yolu olmalıdır — aksi hâlde her dosyadaki her isim, tüm projede görünür (global) hâle gelir ve iki dosya yanlışlıkla aynı ismi kullanırsa çakışma oluşur. Modern JavaScript/TypeScript bu sorunu **ES modülü** sistemiyle çözer: her `.js/.ts` dosyası kendi başına bir modüldür, kendi kapsamını korur; yalnızca `export` anahtar kelimesiyle işaretlenen isimler dışarıdan `import` ile alınabilir.

ES modülü: `import/export` ile dosyalar arasında paylaşılan, kendi kapsamını koruyan kod birimi (*ES module*).

Analoji

ES modülünü bir kütüphaneden ödünç kitap almaya benzetin: her dosya kendi rafını (özel kapsamını) korur; yalnızca “ödünç verilebilir” (`export` edilen) kısımlar başka dosyalarca alınabilir (`import` edilebilir). Rafınızdaki diğer her şey, siz izin vermedikçe kütüphane dışına çıkmaz.

Aşağıdaki iki kesit, bir dosyanın bir fonksiyonu **adlandırılmış** (*named*) bir `export` ile dışarı verdiğini, başka bir dosyanın da bunu `import` ile aldığını gösterir. İlk dosya, `fiyatHesapla` fonksiyonunu `export` anahtar kelimesiyle işaretler:

```
1 export function fiyatHesapla(birimFiyat, adet) {
2   return birimFiyat * adet;
3 }
```

İkinci dosya, bu fonksiyonu süslü parantez içinde — tam olarak hangi ismi istediğini belirterek — `import` eder:

```
1 import { fiyatHesapla } from "../modul-disa-aktar.mjs";
2
3 const toplam = fiyatHesapla(150, 3);
4 console.log("toplam:", toplam);
```

Örnek Olay 3.2: Bir fiyat hesaplama fonksiyonunu dışa/içe aktarma

Dikkat edin: `import` satırındaki `{ fiyatHesapla }` süslü parantezi, ilk dosyadaki `export function fiyatHesapla`’nın adıyla birebir eşleşmelidir — bu, **adlandırılmış** `export`’un doğasıdır (bir dosyanın tek bir `export default`’u da olabilir, ama bu haftaki örnekler yalnızca adlandırılmış `export` kullanır). İki dosya da birbirinden bağımsız kendi kapsamını korur; `fiyatHesapla` ismi yalnızca açıkça `import` edildiği için ikinci dosyada görünür hâle gelir.

3.2.2 Node.js: tarayıcı dışında çalışma zamanı

Az önceki iki dosyayı `.mjs` uzantısıyla ve `node modul-ice-aktar.mjs` komutuyla çalıştırdınız (ya da çalıştıracaksınız) — hiçbir tarayıcı açmadan. Bunu mümkün kılan **Node.js**’tir. Bu dersin geri kalanında kullanacağınız tüm araçlar — `npm`, `vite`, `tsc` — birer JavaScript programıdır ve Node.js üzerinde çalışır. Node.js’i tarayıcının JavaScript motorunu (V8) alıp tarayıcı dışında, düz bir terminal programı gibi çalıştırılabilir hâle getiren bir sarmalayıcı olarak düşünebilirsiniz.

Node.js: tarayıcı dışında JavaScript/TypeScript çalıştırmayı sağlayan çalışma zamanı ortamı (*runtime*).

3.2.3 npm ve package.json: bir projenin kimliği

Paket yöneticisi: bağımlılıkları (kütüphaneleri) indirip sürümlerini yöneten araç (*package manager*).

Paket yöneticisi olan `npm`, Node.js ile birlikte kurulur ve her projenin köküne bir `package.json` dosyası koyar. Bu dosya projenizin kimliğini (ad, sürüm), **bağımlı-**

İkılarını (bir projenin çalışması/derlenmesi için ihtiyaç duyduğu dış paket, *dependency*) ve kısayol komutlarını tek bir yerde tutar. Aşağıda, bu haftanın istemci/projesindeki gerçek `package.json` dosyasını görüyorsunuz:

```

1 {
2   "name": "istemci",
3   "private": true,
4   "version": "0.0.0",
5   "type": "module",
6   "engines": {
7     "node": "^24.13.0 || >=24.13.0"
8   },
9   "scripts": {
10    "dev": "vite",
11    "build": "tsc && vite build",
12    "preview": "vite preview"
13  },
14  "devDependencies": {
15    "typescript": "^7.0.2",
16    "vite": "^8.2.2",
17    "@types/node": "^24.13.3"
18  }
19 }
```

İki bağımlılık türü ayrı tutulur: **dependencies** (bu ders kapsamında bu haftaki projede boş) çalışma zamanında da gereken paketleri tutar; **devDependencies** ise yalnızca **geliştirme/derleme sırasında** gereken, üretime asla gönderilmeyen araçları (burada TypeScript ve Vite’in kendisi) tutar. **scripts** alanındaki üç kısayol — **dev**, **build**, **preview** — terminalde `npm run dev` gibi çağrılır; bu kısayolları birazdan tek tek çalıştıracaksınız.

Analoji

npm ve `package.json`’ı bir markete alışveriş listesiyle gitmeye benzetin: `package.json` alışveriş listesidir (hangi paket, hangi sürüm); npm bu listeye göre kayıt defterinden (*registry*) paketleri toplayıp projenize getirir. Siz `npm install` dediğinizde npm listeyi okur, her paketi indirir ve `node_modules/` klasörüne (marketten eve getirilen ürünler gibi) yerleştirir.

3.2.4 Vite ile proje kurulumu

Bu haftaki istemci/projesi

```
1 npm create vite@latest -- --template vanilla-ts
```

komutuyla oluşturulmuştur. `--template vanilla-ts` bayrağına dikkat edin: bu komut, **saf TypeScript** şablonunu seçer — React şablonunu değil. React, Hf.4’te aynı istemci/klasörüne eklenecektir; bu hafta klasörde henüz hiç React kodu yoktur. Komut çalıştığında ortaya çıkan klasör yapısı şu ana dosyaları içerir: `index.html` (giriş noktası), `src/main.ts` (TypeScript giriş dosyası), `tsconfig.json` (TypeScript derleyici ayarları), `package.json` (az önce gördüğünüz dosya).

Bu hafta `index.html`, Hafta 2’nin `header/nav/ main/footer` iskeletini — **değiştirmeden** — tek bir Vite girişinde birleştirir; yalnızca en alta bir `<script type="module">` etiketi eklenmiştir:

```

1 <!DOCTYPE html>
2 <html lang="tr">
```

```

3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>KTÜ Etkinlik ve Bilet Sistemi</title>
7 </head>
8 <body>
9 <header>
10  <nav class="site-nav">
11    <ul>
12      <li><a href="#etkinlikler">Etkinlikler</a></li>
13      <li><a href="#bilet-al">Bilet Al</a></li>
14    </ul>
15  </nav>
16 </header>
17
18 <main>

```

Sayfanın en altında (</body> kapanışından hemen önce), main.ts'e bağlanan betik etiketi durur:

```
<script type="module" src="/src/main.ts"></script>
```

type="module" özniteliği, tarayıcıya bu betiğin bir ES modülü olduğunu (yani import/export kullanabileceğini) söyler — Vite bu tek girişten başlayarak, main.ts'in içe aktardığı tüm diğer dosyaları otomatik olarak bulup paketler.

tsconfig.json dosyası, TypeScript derleyicisine hedef JavaScript sürümünü, hangi kütüphane tanımlarını (DOM dahil) tanıyacağını ve kod kalitesi denetimlerini ne kadar sıkı uygulayacağını söyler:

```

1 "compilerOptions": {
2   "target": "es2023",
3   "module": "esnext",
4   "lib": ["ES2023", "DOM"],
5   "types": ["vite/client"],
6   "allowArbitraryExtensions": true,
7   "skipLibCheck": true,
8
9   "moduleResolution": "bundler",
10  "allowImportingTsExtensions": true,
11  "verbatimModuleSyntax": true,
12  "moduleDetection": "force",
13  "noEmit": true,
14
15  "noUnusedLocals": true,
16  "noUnusedParameters": true,
17  "erasableSyntaxOnly": true,
18  "noFallthroughCasesInSwitch": true
19 },

```

"noEmit": true önemlidir: bu ayar, tsc'ye "JavaScript dosyası üretme, yalnız tip hatalarını kontrol et" der — gerçek JavaScript'e çevirme işini Vite üstlenir, tsc'nin görevi yalnız denetlemektir. "noUnusedLocals"/"noUnusedParameters" ise kullanılmayan değişken/parametre bıraktığınızda bunu bir hata sayar — unutulmuş kod parçalarını erkenden yakalamanızı sağlar.

İpucu

Bu ders boyunca tsconfig.json'daki bu denetim ayarları korunacaktır. Öğrenciler bazen "kolaylık olsun" diye bu ayarları gevşetmek ister; bunu yapmayın —

ayarlar gevşetildiğinde TypeScript'in size sunduğu erken uyarıların çoğu sessizce kaybolur ve Ünite 3'te göreceğiniz tip hataları editörde artık görünmeyebilir.

3.2.5 Geliştirme sunucusu ve build: iki farklı komut, iki farklı amaç

geliştirme sunucusu:

kod değişikliklerini anlık yansıtan, yalnız geliştirme sırasında kullanılan sunucu (*development server*).

Derleme/yapı (build):

kaynak kodun tip kontrolünden geçirilip optimize edilmiş üretim çıktısına dönüştürülmesi (*build*).

Vite iki tamamen farklı işi ayrı komutlarla yapar. `npm run dev` komutu bir **geliştirme sunucusu** başlatır: `http://localhost:5173` adresinde dinler, kodunuzu her kaydettiğinizde tarayıcıyı — sayfayı elle yenilemenize gerek kalmadan — otomatik günceller (buna *hot module replacement*, HMR denir). Bu sunucu kodu tam olarak derlemez, yalnızca isteneni anlık dönüştürür; hız önceliklidir, tip denetimi arka planda ayrıca çalışır ama sizi asla bloklamaz.

`npm run build` ise tamamen farklı bir işlemi başlatır: **Derleme/yapı (build)** komutu `package.json`'daki `"build": "tsc && vite build"` tanımından da görüldüğü gibi önce `tsc` ile tüm projeyi tip hatalarına karşı denetler (bir tek tip hatası bile varsa `build` başarısız olur), sonra Vite kodu paketleyip (*bundle*) optimize ederek `dist/` klasörüne yazar. Bu `dist/` klasörü, gerçek bir sunucuya (Hf.14'te `nginx`'e) yüklenmeye hazır, saf HTML/CSS/JS dosyalarından oluşur — geliştirme sunucusu asla üretimde kullanılmaz, yalnızca `dist/` çıktısı kullanılır. `npm run preview` ise bu üretim çıktısını yerel makinenizde — sanki gerçek bir sunucudaymış gibi — denemenizi sağlar.

Analoji

Derleme/build sürecini bir mutfakta ham malzemeleri hazır bir yemek kutusuna dönüştürmeye benzetin: kaynak dosyalar (TS/CSS/HTML) tek tek tarayıcıya gönderilmez; Vite bunları "pişirip" tek, optimize edilmiş bir pakete dönüştürür. `npm run dev` ise mutfakta hızlı, anlık bir tadım yapmanıza benzer — yemek henüz kutulanmamıştır ama her malzeme değişikliğini hemen görürsünüz.

3.2.6 Hf.1 bağı: geliştirme sunucusu da bir HTTP sunucusudur

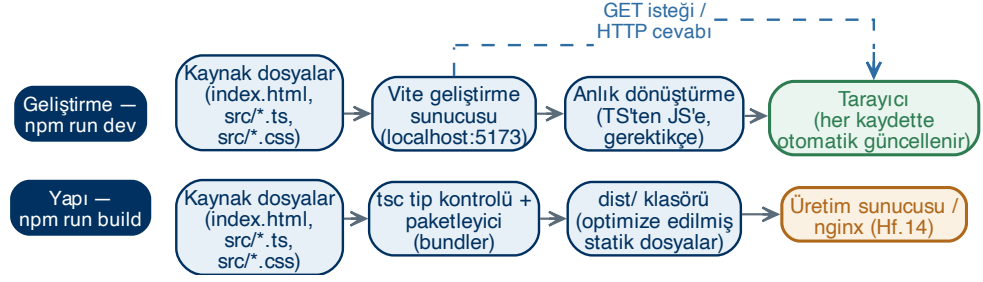
`npm run dev` çalıştırdığınızda terminalde `http://localhost:5173` adresini görürsünüz — bu rastlantı değildir. Vite'in geliştirme sunucusu, Hafta 1 Ünite 2'de öğrendiğiniz standart web sunucusu mimarisinin küçük, yerel bir örneğidir: belirli bir portta (5173) dinler, tarayıcı ona bir `GET` isteği gönderir, o da `index.html` ve modülleri cevap olarak döner. Hafta 1 Ünite 3'ün SPA (*Single Page Application*) kavramı da burada ilk kez somut bir araçla karşılaşır: tek bir `index.html` girişinden, JavaScript ile sayfa içeriği — sayfayı yeniden yüklemekten — güncellenecektir.

Şekil 3.2 bu iki akışı — geliştirme (`npm run dev`) ve yapı (`npm run build`) — kaynak dosyalardan başlayıp farklı hedeflere ulaşan iki ayrı yol olarak gösterir.

📷 EKRAN GÖRÜNTÜSÜ YER TUTUCU

Terminalde `npm run dev` komutu çalıştırıldıktan sonra görünen Vite çıktısı (Local: `http://localhost:5173/` satırı dahil) ve açılan tarayıcı sekmesinde etkinlik listesinin görüldüğü ilk görünüm.

`assets/ss/h03-npm-run-dev-terminal.png`



Şekil 3.2: Vite'in geliştirme akışı (`npm run dev`: kaynak dosyalar → geliştirme sunucusu → anlık dönüştürme → tarayıcı) ile yapı akışı (`npm run build`: kaynak dosyalar → tsc + paketleyici → dist/ → üretim sunucusu).

3 • Katılım Şimdi Siz Deneyin

`npm create vite@latest -- --template vanilla-ts` komutunu çalıştırıp oluşan klasördeki `package.json` dosyasını açın. `scripts` alanındaki `dev` ile `build` komutlarının farkını bir cümleyle açıklayın.

Geribildirim. `dev`, geliştirme sırasında anlık değişiklikleri gösteren, kodu tam derlemeden hızlı geri bildirim veren bir sunucu başlatır (yalnız geliştiricinin makinesinde çalışır); `build` kodu tam olarak tip kontrolünden geçirip optimize ederek `dist/` klasörüne, gerçek bir sunucuya yüklenmeye hazır statik dosyalar üretir.

Kendinizi Sınavın 3.2

`npm run dev` ile `npm run build` arasındaki en önemli iki fark nedir?

5 • İzleme Aktarım — Başka Nerede?

Bu hafta kurduğunuz `kod_ornekleri/hafta03/istemci/` projesi, Hf.4'te React eklenerek büyüyecek; Hf.14'te ise `npm run build` çıktısı (`dist/`) bir Docker imajının içine konup nginx ile gerçek kullanıcılara sunulacaktır — bugün kurduğunuz araç zinciri dönem sonuna kadar aynı kalacaktır.

3.3 TypeScript Tip Sistemi: Etkinlik, Koltuk, Bilet

1 • Ön-öğretim Neden Önemli?

Bir fonksiyona yanlışlıkla bir sayı yerine bir metin gönderdiğinizde JavaScript hiçbir uyarı vermez — hata, kod çalışırken, belki de bir kullanıcının önünde patlar. TypeScript bu hatayı siz kodu yazarken, editörünüzde kırmızı bir çizgiyle gösterir. Bu ünite bu erken uyarı sistemini kurmayı öğretir.

1 • Ön-öğretim

Ön Bilgi

YG I'in kendi `Liste<T>` şablon sınıfını hatırlayın — generic/ template kavramı size zaten tanındıktır. Ünite 1'in dinamik tipleme kavramını da hatırlayın: TypeScript, bu dinamikliğin üstüne, isteğe bağlı ama güçlü bir denetim katmanı ekler; JavaScript'i değiştirmez, ona ek bir güvenlik katmanı giydirir.

Ünite 1'de gördüğünüz gibi, bir JavaScript değişkeninin türü çalışma zamanında değişebilir — bu esneklik bazen avantaj, büyüyen bir projede sıklıkla dezavantajdır. TypeScript, JavaScript'in üstüne yazılan, isteğe bağlı bir tip katmanıdır: yazdığınız `.ts` kodu sonunda yine JavaScript'e dönüşür (Ünite 2'de gördüğünüz `tsc` aracıyla), ama siz kodu yazarken TypeScript editörünüzde — kod hiç çalışmadan — olası hataları işaretler. Bu ünite, Etkinlik/Bilet Sisteminin çekirdek veri tiplerini (Etkinlik, Koltuk, Bilet) tanımlarken bu tip sisteminin temel araçlarını öğretir.

3.3.1 Temel tipler

TypeScript'in en sık kullanacağınız temel tipleri JavaScript'in ilkel tipleriyle birebir örtüşür: `string` (metin), `number` (hem tam sayı hem ondalık — C++'ın aksine TypeScript `int` ile `double`'ı ayırmaz, tek bir `number` tipi vardır), `boolean` (`true/false`). Bir dizi tipi, eleman tipinin sonuna `[]` eklenerek yazılır: `number[]` bir sayı dizisi, `string[]` bir metin dizisidir — C++'ın `std::vector<int>` ile aynı fikri, çok daha kısa bir söz diziminde ifade eder.

İki özel tip dikkat gerektirir. `any` tipi, bir değişkenin herhangi bir tip olabileceğini belirtir — ama bunu yaptığınız anda TypeScript'in o değişken için sağladığı tüm tip denetimini devre dışı bırakırsınız; bu yüzden `any` bu derste neredeyse hiç kullanılmayacaktır. `unknown` ise `any`'nin daha güvenli karşılığıdır: bir değişkenin tipini bilmediğinizi kabul eder, ama onu kullanmadan önce bir tip daraltma kontrolü yapmanızı zorunlu kılar (bu üniteni sonunda tip daraltmayı göreceksiniz).

3.3.2 Tip çıkarımı: her yere tip yazmak zorunda değilsiniz

tip çıkarımı:

derleyicinin bir değişkenin tipini atanan değere bakarak otomatik belirlemesi (*type inference*).

TypeScript'in en pratik özelliklerinden biri **tip çıkarımı**dir: `let sayi = 5;` yazdığınızda, `sayi: number` yazmanıza gerek yoktur — TypeScript atanan 5 değerine bakıp `sayi`'nin tipini otomatik olarak `number` çıkarır ve bu değişkene bundan sonra bir metin atamaya çalışırsanız hata verir. Tip çıkarımı, kodu C++'a göre çok daha az “gürültülü” tutar — her değişken bildirimine tip yazmak zorunda değilsiniz, yalnızca derleyicinin çıkaramayacağı durumlarda (ör. bir fonksiyon parametresi, boş bir dizi) tipi elle belirtirsiniz.

3.3.3 interface ve type: bir nesnenin şeklini tanımlama

Bir nesnenin hangi alanlara ve hangi tiplere sahip olması gerektiğini önceden tanımlamak için **arayüz (interface)** kullanılır.

arayüz (interface):

bir nesnenin sahip olması gereken alan adlarını ve tiplerini tanımlayan yapı (*interface*).

Analoji

`interface/type`'ı bir form şablonuna benzetin: bir başvuru formu hangi alanların (ad, tarih, sayı) hangi tipte doldurulacağını önceden belirler; bir `interface` de bir nesnenin hangi alanlara ve hangi tiplere sahip olması gerektiğini önceden belirler. Şablona uymayan bir form (eksik ya da yanlış tipte bir alan) kabul edilmez.

TypeScript'te aynı işi **type** anahtar kelimesiyle de yapabilirsiniz — ikisi arasındaki pratik fark şudur: **interface** **extends** ile genişletilebilir (bir arayüz başka bir arayüzü miras alabilir), **type** ise birleşim (union) ve kesişim tipleri için daha esnek. Bu haftaki **Etkinlik**, **Koltuk**, **Bilet** nesne şekilleri **interface** ile, durum kümeleri ('bos' | 'secili' | 'dolu' gibi) **type** ile yazılacaktır.

3.3.4 Birleşim (union) ve literal tipler

Bazen bir değişkenin alabileceği değerler, tüm **string**'ler değil, önceden belirlenmiş küçük bir kümedir — ör. bir koltuğun durumu yalnızca “boş”, “seçili” ya da “dolu” olabilir, başka hiçbir metin geçerli değildir. Bu durumu ifade etmek için **birleşim (union) tipi** kullanılır: 'bos' | 'secili' | 'dolu' — burada | işareti “ya da” anlamına gelir. Bu üç değer her biri, tırnak içindeki tam metniyle sabitlenmiş bir **literal tip** (bir değişkenin yalnızca belirli, sabit değerlerden birini alabildiği tip) tir; üçü bir araya gelip **KoltukDurumu** birleşimini oluşturur.

birleşim (union) tipi:
bir değer önceden tanımlı birkaç tipten/değerden yalnızca birine sahip olabilmesi (*union type*).

Analoji

Birleşim tipini bir kapıya iki türden biriyle girebilmeye benzetin (kart ya da şifre): bir değişken önceden tanımlı birkaç olası değerden yalnızca birine sahip olabilir, üçüncü bir seçenek (ör. parmak izi) kabul edilmez — tıpkı 'rezerve' yazmaya çalıştığınızda TypeScript'in bunu reddetmesi gibi.

3.3.5 readonly: ilk atamadan sonra değiştirilemeyen alanlar

Bazı alanların — özellikle bir kimlik (**id**) alanının — bir nesne oluşturulduktan sonra asla değişmemesi gerekir. Bunu belirtmek için **readonly** anahtar kelimesi kullanılır: **readonly id: number;** yazıldığında, bir **Koltuk** nesnesi oluşturulduktan sonra **koltuk.id = 5** gibi bir atama denemesi **tsc** tarafından derleme hatası olarak işaretlenir. Bu, C++'taki bir üye değişkeni yalnızca kurucuda (*constructor*) atayıp sonra **const** yapmaya benzer bir fikirdir — kimlik alanları doğal birer **readonly** adaydır, çünkü bir nesnenin kimliğinin sonradan değişmesi mantıksal bir hatadır.

readonly:
bir alana yalnızca ilk atamada değer verilebileceğini belirten değiştirici (*readonly modifier*).

3.3.6 Generic: Liste<T>

YG I'de kendi **Liste<T>** şablon sınıfınızı yazdınız — bu sınıf, hangi tip için çalışacağını (örn. **Liste<int>** ya da **Liste<std::string>**) kullanım anında belirleyen tek bir tasarımdı. TypeScript'te aynı fikir **generic** adıyla var olur.

generic:
bir tipin/fonksiyonun hangi veri tipiyle çalışacağını kullanım anında parametre olarak alması (*generic*).

Analoji

Generic'i farklı boyuttaki kutulara uyan ayarlanabilir bir rafa benzetin: rafın kendisi tek bir tasarımdır; hangi boyuttaki kutu (hangi tip) konacağı kullanım anında belirlenir — YG I'in **Liste<T>** şablon sınıfı tam olarak bu fikri C++'ta uygulamıştı.

Aşağıdaki kesit, bu haftanın **Liste<T>** generic arayüzünü gösterir — **T**, kullanım anında **Etkinlik** ya da **Koltuk** gibi somut bir tipe değiştirilecek bir yer tutucudur:

```
1 export interface Liste<T> {
2   elemanlar: T[];
3   ekle(eleman: T): void;
4   bul(id: number): T | undefined;
```

```
5 | }
```

`Liste<Etkinlik>` yazdığınızda, TypeScript elemanlar'ın bir `Etkinlik[]` olduğunu, `ekle`'nin bir `Etkinlik` parametresi aldığını ve `bul`'un bir `Etkinlik | undefined` döndürdüğünü otomatik olarak bilir; `Liste<Koltuk>` yazdığınızda ise aynı arayüz, bu kez `Koltuk` tipiyle çalışır — kodu tekrar yazmanıza gerek kalmaz, tıpkı YG I'in `Liste<T>`'sinde olduğu gibi.

3.3.7 Tip daraltma (narrowing)

Bir değişken bir birleşim tipe sahipse (ör. `KoltukDurumu`), o değişkeni kullanmadan önce genellikle hangi alt tipte olduğunu bilmeniz gerekir. Bunu sağlayan mekanizmaya **tip daraltma** denir: `typeof`, eşitlik kontrolü (`===`) ya da `in` operatörü ile bir kontrol yaptığınızda, TypeScript o kontrolün içindeki kod bloğunda değişkenin artık yalnızca o alt tipte olabileceğini “bilir”. Örneğin `if (koltuk.durum === 'dolu')` kontrolünden sonraki blokta TypeScript, `koltuk.durum`'un artık yalnızca `'dolu'` olabileceğini anlar — bu bilgiyi editörünüzün otomatik tamamlaması bile kullanır.

tip daraltma:
bir union tipin, çalışma zamanı kontrolleriyle belirli bir alt tipe indirgenmesi (*type narrowing*).

Analoji

Tip daraltmayı bir zarfı açmadan önce üstündeki pulun ülkesine bakmaya benzetin: zarfın (değişkenin) birden çok olası kaynaktan gelmiş olabileceğini bilirsiniz; pule (kontrole) bakınca hangi ülkeden geldiğini (hangi alt tipte olduğunu) anlar, ona göre davranırsınız.

3.3.8 tsc --noEmit kapısı

Ünite 2'de `tsconfig.json`'da `"noEmit": true` ayarını görmüştünüz. Terminalde doğrudan

```
1 | npx tsc --noEmit
```

çalıştırdığınızda, TypeScript hiçbir JavaScript dosyası üretmeden, projenizdeki tüm `.ts` dosyalarını tip hatalarına karşı toplu biçimde tarar. Bu komut, editördeki kırmızı çizgilerin — bazen gözden kaçabilecek — topluca, tek seferde doğrulanmasıdır; `npm run build`'in ilk adımı da (`tsc && vite build`) tam olarak budur. Bu ders boyunca kodunuzu teslim etmeden önce her zaman tip denetiminin hatasız geçtiğinden emin olacaksınız. (Hafta 4'ten itibaren kullanacağınız Vite react-ts şablonu birden fazla `tsconfig` dosyasını proje referanslarıyla bağlar; orada bu komutun karşılığı `npx tsc -b` olur — bayraksız `tsc --noEmit` o yapıda hiçbir dosyayı denetlemez.)

3.3.9 Alan modeli tipleri: Etkinlik, Koltuk, Bilet

Şimdi bu ünitenin tüm araçlarını (interface, union/literal, readonly, generic) bir araya getirerek Etkinlik/Bilet Sisteminin çekirdek alan modelini tanımlayın — bu üç tip, Hf.4'ten Hf.7'ye kadar aynen kullanılacaktır, bir daha değişmeyecektir. İlk tip, `Etkinlik`:

```
1 | export interface Etkinlik {
2 |   id: number;
3 |   baslik: string;
4 |   tarih: string;
5 |   mekan: string;
```

```

6   kapasite: number;
7   biletFiyati: number;
8 }

```

`biletFiyati` alanına dikkat edin: bu alan Hafta 2'nin statik tablosunda yoktu — bu hafta bilinçli olarak eklendi, çünkü Ünite 4'te koltuk seçiminde “toplam tutar” hesaplayacaksınız, Hf.5'te bir sepet oluşacaktır. İkinci tip, `Koltuk` — burada hem `readonly` hem birleşim/literal tipi bir arada görürsünüz:

```

1 export type KoltukDurumu = 'bos' | 'secili' | 'dolu';
2
3 export interface Koltuk {
4   readonly id: number;
5   readonly etkinlikId: number;
6   sıra: number;
7   koltukNo: number;
8   durum: KoltukDurumu;
9 }

```

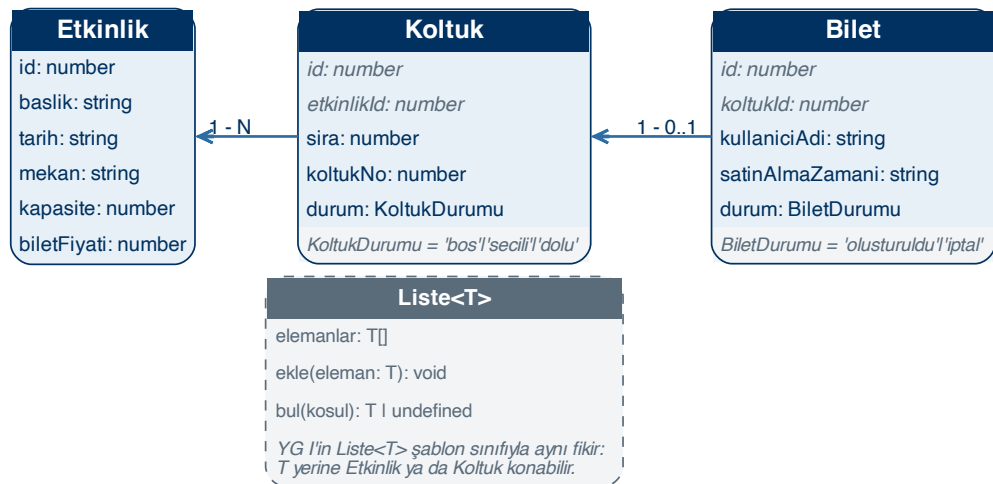
`id` ve `etkinlikId` `readonly`'dır — bir koltuğun kimliği ve hangi etkinliğe ait olduğu, oluşturulduktan sonra asla değişmez; `sıra`, `koltukNo`, `durum` ise değiştirilebilir (özellikle `durum`, Ünite 4'te koltuğa tıklandıkça değişecektir). Üçüncü tip, `Bilet`:

```

1 export type BiletDurumu = 'olusturuldu' | 'iptal';
2
3 export interface Bilet {
4   readonly id: number;
5   readonly koltukId: number;
6   kullanıcıAdi: string;
7   satınAlmaZamani: string;
8   durum: BiletDurumu;
9 }

```

Şekil 3.3 bu üç tipi, alanlarını ve aralarındaki ilişki çokluğunu (bir etkinliğin birden çok koltuğu, bir koltuğun en fazla bir bileti olabileceğini) birlikte gösterir.



Şekil 3.3: Etkinlik/Bilet Sisteminin alan modeli: Etkinlik, Koltuk, Bilet arayüzleri ve aralarındaki ilişki çokluğu (1-N, 1-0..1); italik yazılmış ve gri alanlar `readonly` olarak tanımlıdır.

Bu tipler soyut kalmaz — `veri.ts` dosyasında bu tiplere uygun, somut bir veri dizisiyle örneklenirler. Dizin ilk iki nesnesi bu kalıbı gösterir:

```

1 {
2   id: 1,
3   baslik: 'Bahar Konseri',
4   tarih: '2026-05-15',
5   mekan: 'Açık Hava Amfisi',
6   kapasite: 500,
7   biletFiyati: 150,
8 },
9 {
10  id: 2,
11  baslik: 'Kariyer Günleri',
12  tarih: '2026-05-22',
13  mekan: 'Kongre Merkezi',
14  kapasite: 800,
15  biletFiyati: 0,
16 },

```

Üçüncü nesne (`id: 3`, “Tiyatro Gecesi”) aynı alanlarla, `veri.ts` dosyasının devamında tanımlıdır; dizinin tamamı üç etkinlik içerir. Koltuk ızgarası da aynı dosyada, aynı yöntemle kurulur:

```

1 export const koltuklar: Koltuk[] = [
2   { id: 1, etkinlikId: 1, sıra: 1, koltukNo: 1, durum: 'bos' },
3   { id: 2, etkinlikId: 1, sıra: 1, koltukNo: 2, durum: 'bos' },
4   { id: 3, etkinlikId: 1, sıra: 1, koltukNo: 3, durum: 'bos' },
5   { id: 4, etkinlikId: 1, sıra: 1, koltukNo: 4, durum: 'bos' },
6   { id: 5, etkinlikId: 1, sıra: 2, koltukNo: 1, durum: 'bos' },
7   { id: 6, etkinlikId: 1, sıra: 2, koltukNo: 2, durum: 'dolü' },
8   { id: 7, etkinlikId: 1, sıra: 2, koltukNo: 3, durum: 'bos' },
9   { id: 8, etkinlikId: 1, sıra: 2, koltukNo: 4, durum: 'bos' },
10  { id: 9, etkinlikId: 1, sıra: 3, koltukNo: 1, durum: 'bos' },
11  { id: 10, etkinlikId: 1, sıra: 3, koltukNo: 2, durum: 'bos' },
12  { id: 11, etkinlikId: 1, sıra: 3, koltukNo: 3, durum: 'bos' },
13  { id: 12, etkinlikId: 1, sıra: 3, koltukNo: 4, durum: 'dolü' },
14 ];

```

Dikkat edin: `etkinlikler` dizisindeki her nesne, `Etkinlik` arayüzünün tüm alanlarını doğru tiplerle içermek zorundadır — bir alanı eksik bırakırsanız ya da yanlış tipte bir değer verirsiniz `tsc` bunu derleme hatası olarak işaretler. Bu, Hafta 2’de elle HTML’e yazdığınız etkinlik verisinin artık **tip güvenli** bir TypeScript dizisine dönüştüğü andır.

3 • Katılım Şimdi Siz Deneyin

Aşağıdaki `Koltuk` arayüzünü tanımlayın: `id` (salt okunur sayı), `etkinlikId` (salt okunur sayı), `sıra` (sayı), `koltukNo` (sayı), `durum` (`'bos' | 'secili' | 'dolü'`). Sonra bir `Koltuk` nesnesi oluşturup `durum` alanına `'rezerve'` atamayı deneyin — ne olur?

Geribildirim. Arayüz şu biçimde tanımlanır:

```

interface Koltuk {
  readonly id: number;
  readonly etkinlikId: number;
  sıra: number;
  koltukNo: number;
  durum: 'bos' | 'secili' | 'dolü';
}

```

`durum = 'rezerve'` ataması `tsc` derleyicisinde hata verir, çünkü `'rezerve'` union tipinde tanımlı üç değerden biri değildir — `tsc --noEmit` bu hatayı derleme/çalıştırma adımından önce, editördeki yakalar.

Kendinizi Sınavın 3.3

Koltuk tipinizde `durum`: `'bos' | 'secili' | 'dolu'` tanımlıyken kodun bir yerinde `koltuk.durum = 'rezerve'` yazarsanız ne olur ve bu neden faydalıdır?

5 • İzleme Aktarım — Başka Nerede?

Bu ünite tanımladığınız `Etkinlik/Koltuk/Bilet` arayüzleri Hf.4'te React bileşenlerinin `props` tipleri olarak, Hf.6'da FastAPI'nin Pydantic modelleriyle aynı alanları taşıyan bir “uçtan uca tip hikâyesi”nin TypeScript ucu olarak aynen kullanılacaktır — bugün yazdığınız tip tanımları dönem boyunca değişmeyecektir.

3.4 Asenkron JavaScript ve DOM: Olay Döngüsü, Promise, async/await

1 • Ön-öğretim Neden Önemli?

Hafta 2'de yazdığınız koltuk planındaki koltuklara tıkladığında hiçbir şey olmuyordu; bilet formunda “Gönder”e basınca sayfa boşuna yenileniyordu. Bu ünite sonunda, aynı HTML'e dokunmadan, üstüne yazdığınız TypeScript ile o sayfayı ilk kez “canlı” hâle getireceksiniz.

1 • Ön-öğretim Ön Bilgi

Hafta 2'nin `koltuk-planı/bilet-formu` etiketli HTML yapısını hatırlayın — bu hafta HTML'in kendisi değişmeden korunur. Ünite 3'ün `Etkinlik/Koltuk/Bilet` tiplerini de hatırlayın — bu ünite onları gerçek bir dizi veriye, ekranda görünen ve tıklanabilen bir arayüze dönüştürür.

Ünite 1–3'te JavaScript'in dilini, projenizi paketleyen araçları ve tip sistemini öğrendiniz. Bu son ünite, hepsini birleştirerek Hafta 2'nin hareketsiz sayfasına ilk kez **davranış** ekler: bir `Etkinlik[]` dizisinden dinamik bir kart listesi üretmeyi, koltuklara tıklama olaylarını dinlemeyi ve bir bilet formunun gönderimini — sayfayı yenilemeden — ele almayı öğreneceksiniz. Ama önce, bu etkileşimlerin JavaScript'te **ne zaman** çalıştığını yöneten temel mekanizmayı — olay döngüsünü — anlamanız gerekir.

3.4.1 Olay döngüsü: JavaScript'in tek iş parçacığı

YG II'de `std::thread` ile gerçek paralel iş parçacıkları açtınız — birden fazla kod parçası gerçekten aynı anda çalışabiliyordu. JavaScript bunun tam tersini yapar: **olay döngüsü**, tek bir iş parçacığında çalışır — aynı anda yalnızca bir kod parçası yürütülebilir. “Eşzamanlılık” burada gerçek paralellik değildir, bekleyen işlerin bir sıraya girip tek tek işlenmesiyle sağlanır.

olay döngüsü:
çağrı yığını boşaldığında
bekleyen görevleri
sırayla işleyen
JavaScript mekanizması
(*event loop*).

Bunu anlamak için üç yapıyı bilmeniz gerekir. Birincisi **çağrı yığını** (o an çalışmakta olan fonksiyon çağrılarının tutulduğu yapı, *call stack*)'dır — bir fonksiyon çağrıldığında yığına eklenir, bittiğinde yığından çıkarılır; C++'taki fonksiyon çağrı yığınıyla aynı fikri taşır. İkincisi, gecikmeli ya da asenkron işlerin (bir `setTimeout`'un süresi dolduğunda, bir ağ isteğinin cevabı geldiğinde) beklediği **görev kuyruğudur**. Kural nettir ve sıkça yanlış anlaşılır: **çağrı yığını tamamen boşalmadan**, görev kuyruğundaki hiçbir iş alınmaz — `setTimeout(fn, 0)` bile “hemen” değil, “sıra gelince” anlamına gelir.

görev kuyruğu:
olay döngüsünün çağrı yığını boşalınca sırayla işlediği bekleyen işler listesi (*task queue*).

Üçüncüsü, iki ayrı kuyruk vardır ve öncelikleri farklıdır: bir **mikro görev kuyruğu** (`Promise` callback'leri) ile bir **makro görev kuyruğu** (`setTimeout` callback'leri). Olay döngüsü, çağrı yığını her boşaldığında önce mikro görev kuyruğunu tamamen boşaltır, ancak ondan sonra makro görev kuyruğundan bir iş alır. Aşağıdaki kesit bu sırayı doğrudan gösterir:

```
1 console.log("A");
2 setTimeout(() => console.log("B"), 0);
3 Promise.resolve().then(() => console.log("C"));
4 console.log("D");
```

Adım adım izleyin: `console.log("A")` senkron kod olduğu için hemen çalışır ve A yazdırılır. `setTimeout(..., 0)` çağrısı callback'ini **makro** görev kuyruğuna koyar, hemen çalıştırmaz. `Promise.resolve().then(...)` callback'ini **mikro** görev kuyruğuna koyar. `console.log("D")` yine senkron kod olduğu için hemen çalışır, D yazdırılır. Bu noktada çağrı yığını boşalmıştır — olay döngüsü devreye girer: önce mikro kuyruktaki C işlenir, ancak sonra makro kuyruktaki B işlenir. Sonuç sırası: A, D, C, B.

Analoji

Olay döngüsünü tek kasiyerli bir markette bekleme sırasına benzetin: kasiyer (JS'in tek iş parçacığı) aynı anda yalnızca bir müşteriyle ilgilenir; gecikmeli işler (`setTimeout`, bir isteğin cevabı) bir bekleme sırasına alınır, kasiyer eldeki müşteriyi bitirmeden sıradakine geçmez — “0 saniye sonra gel” diyen bir müşteri bile, kasiyer eldeki işi bitirene kadar beklemek zorundadır.

Şekil 3.4 çağrı yığını, tarayıcının sağladığı Web API'ler, mikro/makro görev kuyrukları ve olay döngüsü arasındaki akışı, mikro kuyruğun her zaman önce boşaltıldığını vurgulayarak gösterir.

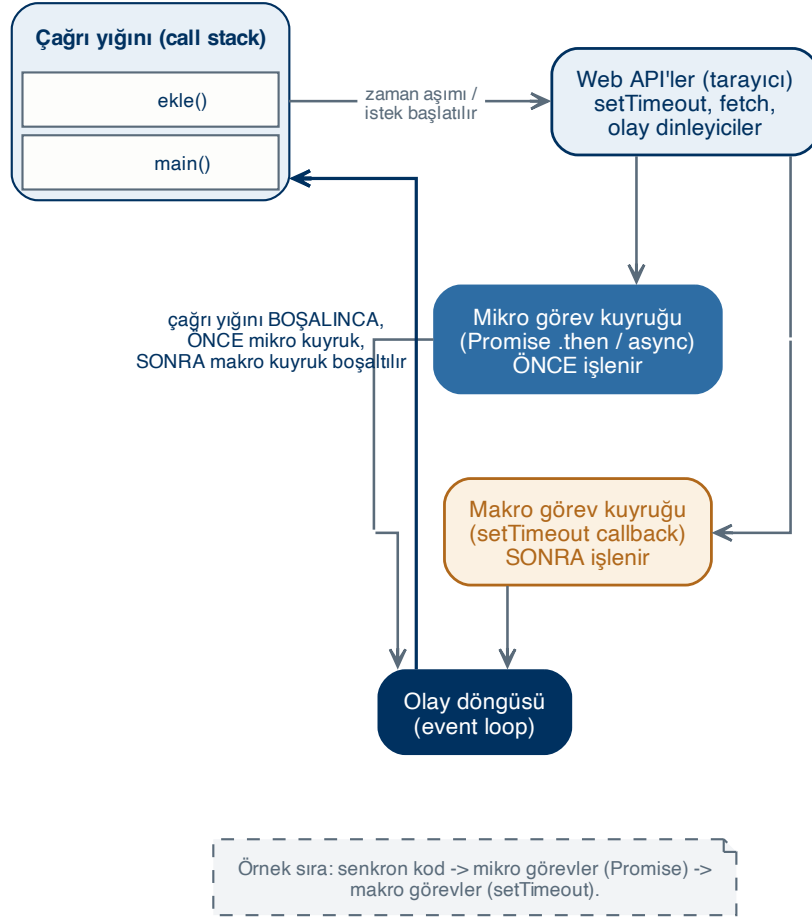
Ortak Hata / Tuzak

Yeni başlayanlar sıklıkla `setTimeout(fn, 0)`'ın fonksiyonu **hemen** çalıştıracak sanır — “0 milisaniye” ifadesi buna yol açar. Oysa 0 yalnızca “en kısa gecikmeyle kuyruğa gir” demektir; fonksiyon yine de mevcut senkron kod tamamen bitene (hatta tüm mikro görevler işlenene) kadar çalışmaz. “0 ms” bile “sırada beklemek” anlamına gelir.

3.4.2 Promise: ileride sonuçlanacak bir söz

Bir ağ isteği ya da gecikmeli bir işlem başlattığınızda, sonucu hemen elinizde olmaz. JavaScript bu durumu bir **Promise** nesnesiyle temsil eder: `.then()` ile işlem başarıyla sonuçlandığında çalışacak kodu, `.catch()` ile işlem hatayla sonuçlandığında çalışacak kodu zincirlersiniz.

Promise:
bir asenkron işlemin ileride başarı ya da hatayla sonuçlanacağını temsil eden nesne (*Promise*).



Şekil 3.4: Çağrı yığını, tarayıcının Web API'leri, mikro görev kuyruğu (Promise) ve makro görev kuyruğu (setTimeout) arasındaki akış; mikro kuyruk her zaman makro kuyruktan önce boşaltılır.

Analoji

Promise'ı bir kuru temizlemeciye bırakılan makbuza benzetin: işiniz hemen bitmez ama elinizdeki makbuz (Promise) size ileride ya “hazır” ya da “sorun çıktı” bilgisini kesin olarak verecektir — makbuzu elinize aldığınız an sonucu bilmezsiniz, ama bir gün mutlaka öğrenirsiniz.

```

1 function biletOlusturSoz(ad) {
2   return new Promise((resolve) => {
3     setTimeout(() => {
4       resolve(`${ad} için bilet oluşturuldu`);
5     }, 200);
6   });
7 }
8
9 console.log("[then] istek gönderildi");
10 biletOlusturSoz("Ayşe")
11   .then((sonuc) => {
12     console.log("[then] sonuç:", sonuc);
13   })
14   .catch((hata) => {
15     console.error("[then] hata:", hata);
16   });

```

3.4.3 async/await: senkron görünümlü asenkron kod

`.then()` zincirleri, birden çok asenkron adım art arda geldiğinde okunması zorlaşan iç içe geçmiş kod üretebilir. Modern JavaScript, aynı Promise zincirini senkron görünümlü bir söz diziminde yazmanıza izin verir: bir fonksiyonu `async` olarak işaretlersiniz, içinde bir Promise'in sonucunu `await` ile “beklersiniz”. Kritik ayrıntı şudur: `await` yalnızca içinde bulunduğu `async` fonksiyonu duraklatır, tüm programı bloklamaz — olay döngüsü bu bekleme sırasında başka işleri (ör. bir düğmeye tıklama olayını) işlemeye devam edebilir.

Aşağıdaki kesit, tam olarak aynı “bilet oluşturma” işlemini önce `.then()` ile (yukarıda gördünüz), şimdi de `async/await` ile yazar:

```

1 async function biletOlusturAsync(ad) {
2   console.log("[async] istek gönderildi");
3   const sonuc = await biletOlusturSoz(ad);
4   console.log("[async] sonuç:", sonuc);
5   return sonuc;
6 }
7
8 biletOlusturAsync("Mehmet");

```

Örnek Olay 3.3: Aynı gecikmeli işlemi iki yazımla karşılaştırma

İki yazım da tamamen aynı şeyi yapar — `biletOlusturSoz` fonksiyonunu çağırıp sonucunu bekler. Farkın nerede olduğuna dikkat edin: `.then()` yazımında sonucu işleyen kod ayrı bir callback fonksiyonunun içindedir; `async/await` yazımında ise aşağıdaki satır, sanki senkron bir fonksiyon çağırısıymış gibi sonucu doğrudan bir değişkene atar:

```
const sonuc = await biletOlusturSoz(ad);
```

Kod yukarıdan aşağıya, ekstra girinti (*indentation*) katmanı olmadan okunur. Bu ders boyunca — ve az sonra biletle formunda — `async/await` tercih edilecektir, çünkü okunabilirliği daha yüksektir.

3.4.4 setTimeout: gecikmeli bir işi kuyruğa alma

`setTimeout(fn, gecikme)` çağrısı, `fn` fonksiyonunu hemen çalıştırmaz — `gecikme` milisaniye geçtikten sonra makro görev kuyruğuna ekler (olay döngüsü kuralları burada da geçerlidir). Bu ünite de bu mekanizma, bir sunucuya biletle oluşturma isteği göndermenin — Hf.6'ya kadar gerçek bir sunucu olmadığı için — gecikmesini simüle etmek için kullanılır: `biletOlustur` fonksiyonu, tıpkı gerçek bir ağ isteği gibi, sonucu bir süre sonra döndüren bir `Promise` sarmalar.

3.4.5 DOM API: sayfayı JavaScript ile değiştirme

DOM:
bir HTML belgesinin
JavaScript ile erişilip
değiştirilebilen ağaç
yapısındaki temsili
(*Document Object Model*).

Şimdiye kadar gördüğünüz her şey saf JavaScript diliyle ilgiliydi; şimdi bu dili, tarayıcının size sunduğu **DOM** API'siyle birleştireceksiniz. Üç temel işlem yeterlidir: `document.querySelector('#id')` ya da `document.querySelectorAll('.sınıf')` bir ya da birden çok HTML öğesini seçer; `element.addEventListener('click', fn)` bir öğeye bir olay dinleyicisi ekler (`fn` o olay — tıklama, form gönderimi — gerçekleştiğinde çağrılır); `document.createElement('div')` ile yeni bir öğe oluşturup `element.appendChild(cocuk)` ile sayfaya eklersiniz.

Analoji

DOM'u bir binanın güncellenebilir kat planına benzetin: `querySelector` planda belirli bir odayı bulur; `addEventListener` o odaya "biri kapıyı çaldığında ne olacağını" önceden tanımlar; `createElement` plana yeni bir oda eklemek gibidir.

3.4.6 Hafta 2 sayfasını canlandırma

Şimdi tüm bu araçları birleştirerek Hafta 2'nin dört durağan bölümünü — etkinlik listesi, koltuk planı, seçim özeti, biletle formu — sırayla "canlandırın". İlk olarak, etkinlik listesi artık HTML'e elle yazılmaz; `etkinlikler` dizisinden `forEach` ile DOM üzerinde üretilir:

```
1 export function etkinlikListesiUret(
2   kapsayici: HTMLElement,
3   etkinlikler: Etkinlik[]
4 ): void {
5   kapsayici.innerHTML = '';
6
7   etkinlikler.forEach((etkinlik) => {
```

Kapsayıcı önce boşaltılır, ardından dizideki her etkinlik için `forEach` bir döngü adımı çalıştırır. Döngünün her adımında `createElement` ile Hafta 2'nin `article` tabanlı etkinlik kartı yapısı yeniden oluşturulur:

```
1 const kart = document.createElement('article');
2 kart.className = 'etkinlik-karti';
3
4 const baslik = document.createElement('h3');
5 baslik.textContent = etkinlik.baslik;
6
```

```

7 const bilgi = document.createElement('p');
8 bilgi.textContent = `${etkinlik.tarih} · ${etkinlik.mekan}`;
9
10 const kapasite = document.createElement('p');
11 kapasite.textContent = `Kapasite: ${etkinlik.kapasite} · Bilet: ${etkinlik.
    biletFiyati} TL`;
12
13 kart.append(baslik, bilgi, kapasite);
14 kapsayici.appendChild(kart);

```

Başlık, bilgi ve kapasite/bilet satırları ayrı `p/h3` öğeleri olarak oluşturulup `append` ile karta, kart da `appendChild` ile kapsayıcıya eklenir — HTML elle yazılmaz, aynı yapı kod ile üretilir; bu, dizideki etkinlik sayısı değişse bile (3 yerine 30 etkinlik olsa bile) kodun tek satır bile değişmeden çalışmaya devam etmesini sağlar.

İkinci olarak, koltuk planı — her koltuk kendi `click` dinleyicisiyle üretilir. Dış çerçeve etkinlik listesininkiyle aynı kalıbı izler: kapsayıcı boşaltılır, dizi üzerinde `forEach` ile gezilir:

```

1 export function koltukPlaniUret(
2   kapsayici: HTMLElement,
3   koltuklar: Koltuk[],
4   degisiklikOldu: () => void
5 ): void {
6   kapsayici.innerHTML = '';
7
8   koltuklar.forEach((koltuk) => {

```

Her koltuk için tek bir `div.koltuk` öğesi oluşturulur, dolu koltuklara `dolu` sınıfı eklenir ve öğeye bir `click` dinleyicisi bağlanır:

```

1 const eleman = document.createElement('div');
2 eleman.className = 'koltuk';
3 eleman.textContent = koltukEtiketi(koltuk);
4
5 if (koltuk.durum === 'dolu') {
6   eleman.classList.add('dolu');
7 }
8
9 eleman.addEventListener('click', () => {
10   koltukTiklamaIsleyici(koltuk, eleman, degisiklikOldu);
11 });
12
13 kapsayici.appendChild(eleman);

```

Bir koltuğa tıklandığında hangi mantığın çalışacağını ayrı bir fonksiyon belirler:

```

1 function koltukTiklamaIsleyici(
2   koltuk: Koltuk,
3   eleman: HTMLDivElement,
4   degisiklikOldu: () => void
5 ): void {
6   if (koltuk.durum === 'dolu') {
7     return;
8   }
9
10  koltuk.durum = koltuk.durum === 'bos' ? 'secili' : 'bos';
11  eleman.classList.toggle('secili', koltuk.durum === 'secili');
12  degisiklikOldu();
13 }

```

Bu fonksiyon Ünite 3'ün `KoltukDurumu` birleşim tipini fiilen kullanır: `koltuk` 'dolu' ise fonksiyon hiçbir şey yapmadan geri döner (dolu koltuklar tıklanamaz); aksi hâlde durum 'bos' ↔ 'secili' arasında geçiş yapar ve `classList.toggle` ile CSS sınıfını günceller — `style.css` dosyasındaki `.koltuk.secili/.koltuk.dolu` kuralları bu sınıfa göre rengi değiştirir:

```
1 .koltuk.secili {
2   background-color: var(--renk-yesil);
3   color: #FFFFFF;
4   border-color: var(--renk-yesil);
5 }
6
7 .koltuk.dolu {
8   background-color: var(--renk-acikgri);
9   border-color: var(--renk-gri);
10  color: var(--renk-gri);
11  cursor: not-allowed;
12 }
```

Üçüncü olarak, seçili koltuk sayısı ve toplam tutar — her koltuk tıklamasından sonra — yeniden hesaplanır:

```
1 export function secimOzetiGuncelle(
2   eleman: HTMLElement,
3   koltuklar: Koltuk[],
4   biletFiyati: number
5 ): number {
6   const seciliSayisi = koltuklar.filter((k) => k.durum === 'secili').length;
7   const toplamTutar = seciliSayisi * biletFiyati;
8
9   eleman.textContent = `Seçili koltuk: ${seciliSayisi}, Toplam: ${toplamTutar}
10  } TL`;
11
12   return seciliSayisi;
13 }
```

Bu fonksiyon Ünite 1'in `filter` metodunu kullanır: yalnızca `durum === 'secili'` olan koltukları sayar, toplam tutarı `seciliSayisi * biletFiyati` ile hesaplar. Son olarak, bilet formunun `submit` olayında sayfanın yenilenmesi engellenir ve seçim özeti, gecikmeli bir `async` çağrıyla “sunucuya gönderilmiş gibi” işlenir:

```
1 async function biletOlustur(ozet: SecimOzeti): Promise<Bilet> {
2   return new Promise((resolve) => {
3     setTimeout(() => {
4       resolve({
5         id: Date.now(),
6         koltukId: ozet.koltukIdleri[0] ?? 0,
7         kullaniciAdi: ozet.kullaniciAdi,
8         satinAlmaZamani: new Date().toISOString(),
9         durum: 'olusturuldu',
10       });
11     }, 1000);
12   });
13 }
```

Formun `submit` dinleyicisi, önce varsayılan davranışı iptal eder:

```
1 export function biletFormBaglan(
2   form: HTMLFormElement,
3   adSoyadGirdi: HTMLInputElement,
4   koltuklar: Koltuk[]
```

```

5 ): void {
6   form.addEventListener('submit', (olay) => {
7     olay.preventDefault();

```

`olay.preventDefault()` çağrısı, formun varsayılan davranışını — sayfayı yeniden yükleyip sunucuya bir HTTP isteği gönderme davranışını — iptal eder; bunsuz Hafta 2’de gördüğünüz gereksiz sayfa yenilenmesi devam ederdi. Ardından seçili koltuklar toplanıp bir özet nesnesi kurulur:

```

1 const seciliKoltuklar = koltuklar.filter((k) => k.durum === 'secili');
2 const ozet: SecimOzeti = {
3   kullanıcıAdi: adSoyadGirdi.value,
4   koltukIdleri: seciliKoltuklar.map((k) => k.id),
5 };
6
7 console.log('Bilet özeti:', ozet);

```

Seçili koltuklar `filter` ile toplanır, kullanıcı adı ve koltuk kimlikleri `ozet` nesnesine yerleştirilir, konsola bir özet yazılır. Son olarak `biletOlustur` çağrılır:

```

1 biletOlustur(ozet).then((bilet) => {
2   console.log('Bilet oluşturuldu:', bilet);
3 });

```

Bu çağrı — yaklaşık 1 saniyelik bir gecikmeyle — “bilet oluşturuldu” onayını konsola basar. `biletFormBaglan` fonksiyonunun tamamı (dinleyiciyi kapatan parantezlerle birlikte) `kod_ornekleri/hafta03/istemci/src/biletForm.ts` dosyasındadır.

Tüm bu parçaları birbirine bağlayan giriş noktası `src/main.ts`’tir:

```

1 const etkinlikListesiElemani = document.querySelector<HTMLDivElement>('#
   etkinlik-listesi')!;
2 const koltukPlaniElemani = document.querySelector<HTMLDivElement>('#koltuk-
   plani')!;
3 const secimOzetiElemani = document.querySelector<HTMLParagraphElement>('#
   secim-ozeti')!;
4 const biletFormuElemani = document.querySelector<HTMLFormElement>('#bilet-
   formu')!;
5 const adSoyadElemani = document.querySelector<HTMLInputElement>('#ad-soyad')
   !;
6
7 const seciliEtkinlik = etkinlikler.find((e) => e.id === koltuklar[0].
   etkinlikId)!;
8
9 function ozetiYenile(): void {
10   secimOzetiGuncelle(secimOzetiElemani, koltuklar, seciliEtkinlik.biletFiyati
       );
11 }
12
13 etkinlikListesiUret(etkinlikListesiElemani, etkinlikler);
14 koltukPlaniUret(koltukPlaniElemani, koltuklar, ozetiYenile);
15 biletFormBaglan(biletFormuElemani, adSoyadElemani, koltuklar);
16
17 ozetiYenile();

```

📷 EKRAN GÖRÜNTÜSÜ YER TUTUCU

Tarayıcıda çalışan sayfada birkaç koltuğa tıklanmış (rengi “seçili” durumuna dönmüş), seçim özetinin (“Seçili koltuk: N, Toplam: X TL”) güncellendiği ve Geliştirici Araçları Konsol sekmesinde bilet formu gönderildikten sonra görünen özet nesnesi ile “Bilet oluşturuldu” onay logunun bir arada görüldüğü an.

assets/ss/h03-koltuk-secimi-canli-sayfa.png

3.4.7 React’e geçiş gerekçesi

Az önce yazdığınız kodu bir kez daha gözden geçirin: her koltuğa elle bir `addEventListener` eklediniz, listeyi elle `createElement` ile ürettiniz, seçim özetini her tıklamadan sonra elle yeniden hesapladınız. On iki koltuk için bu yönetilebilir; ama koltuk/etkinlik sayısı yüzlere çıktığında, ya da sayfada aynı anda güncellenmesi gereken onlarca küçük parça olduğunda, bu elle senkronizasyon hızla karmaşıklaşan, hataya açık bir iş hâline gelir — bir yeri güncellemeyi unutursanız ekran ile gerçek veri birbirinden sapar. Hf.4’te öğreneceğiniz React, tam olarak bu senkronizasyonu — verideki bir değişikliği fark edip ekranı otomatik güncellemeyi — sizin yerinize üstlenecektir.

3 • Katılım 📝 Şimdi Siz Deneyin

Aşağıdaki kodun konsola hangi sırada (A, B, C, D) yazacağını tahmin edin, sonra çalıştırıp doğrulayın:

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
```

Geribildirim. Sıra **A, D, C, B**’dir — senkron kod (A, D) önce tamamen biter; sonra mikro görev kuyruğundaki **Promise** callback’i (C) çalışır; en son makro görev kuyruğundaki **setTimeout** (B) çalışır — **setTimeout(fn, 0)** bile “hemen” değil “sıra bana gelince” demektir.

📝 Kendinizi Sınavın 3.4

`setTimeout(fn, 0)` çağrısı neden `fn`’i hemen değil, mevcut senkron kod bittikten sonra çalıştırır?

5 • İzleme ➡ Aktarım — Başka Nerede?

Bu üniteye yazdığınız koltuk tıklama ve form gönderim mantığı, Etkinlik/Bilet Sisteminin ilk davranış katmanıdır; Hf.4’te bu elle yazılmış DOM güncelleme kodu, React’in `state/onClick` ile otomatikleştirdiği aynı işi temsil edecek — “elle yapılan” ile “React’in sizin yerinize yaptığı” arasındaki fark orada somutlaşacaktır.

Bölüm Özeti

Özet

Bu hafta, Hafta 2'nin hareketsiz bıraktığı statik sayfaya ilk kez davranış eklediniz. İlk ünite de JavaScript'in dil modelini öğrendiniz: ilkel tiplerin kopyalanarak, nesne/dizi tiplerinin ise ortak bir belleğe referansla atandığını; fonksiyon bildirimi, fonksiyon ifadesi ve ok fonksiyonu arasındaki farkları; bir iç fonksiyonun kapandığı kapsama — closure ile — erişmeye devam ettiğini; her nesnenin bir prototipi olduğunu ve `class` söz diziminin bu prototip mekanizmasının üstüne kurulmuş bir kısayol olduğunu gördünüz. İkinci ünite de bu dili çalıştırılabilir bir projeye dönüştüren araç zincirini kurdunuz: ES modülleriyle kodu dosyalara böldünüz, npm ve `package.json` ile bağımlılıkları yönettiniz, Vite ile bir TypeScript projesi oluşturup `npm run dev` (geliştirme) ile `npm run build` (üretim) arasındaki farkı deneyimlediniz. Üçüncü ünite de TypeScript'in tip sistemini kullanarak Etkinlik/Bilet Sisteminin çekirdek veri tiplerini — `Etkinlik`, `Koltuk`, `Bilet` — `interface`, birleşim/literal tipler, `readonly` ve `Liste<T>` generic'iyle tanımladınız; bu tanımlar Hf.4'ten Hf.7'ye kadar değişmeden kullanılacaktır. Son ünite de olay döngüsünü — çağrı yığını, mikro/makro görev kuyrukları — Promise'i, `async/await`'i ve DOM API'sini birleştirerek Hafta 2'nin sayfasını ilk kez canlandırdınız: etkinlik listesi artık bir diziden üretiliyor, koltuklar tıklanabiliyor, seçim özeti anlık güncelleniyor, bilet formu sayfayı yenilemeden bir özet basıyor. Gelecek hafta, bu elle yazılmış DOM güncelleme kodunun neden hızla karmaşılaştığını gördükten sonra, React ile aynı işi bileşenlere bölerek çok daha sürdürülebilir bir biçimde yapacaksınız.

Kendinizi Sınavın Cevapları

Soru (Ünite 1 — `console.log(sayilar)` çıktısı). Çıktı `[1, 2, 3, 99]`'dur. Nedeni, dizinin fonksiyona bir **referans** olarak geçirilmesidir: `ekle(sayilar)` çağrısı `ekle` fonksiyonunun içindeki `dizi` parametresini, dışarıdaki `sayilar` değişkeniyle aynı diziye bağlar — ikisi artık tek bir diziyi işaret eden iki farklı isimdir. Fonksiyon içindeki `dizi.push(99)` çağrısı bu ortak diziyi mutasyona uğratar; fonksiyon bittikten sonra dışarıdaki `sayilar` değişkeninden de bu değişiklik görünür, çünkü kopyalanmış ayrı bir dizi hiç oluşmamıştır. Bu davranış, C++'ta bir diziyi/vektörü referansla (&) bir fonksiyona geçirmeye birebir karşılık gelir — JavaScript'te nesne ve dizi parametreleri için "değere göre geçirme" seçeneği zaten yoktur, her zaman referans olarak geçerler.

Soru (Ünite 2 — `npm run dev` ile `npm run build` farkı). İki en önemli fark şudur: birincisi **amaç** farkıdır — `dev` anlık/hot-reload çalışan bir geliştirme sunucusu başlatır, kodu tam derlemeden hızlı geri bildirim verir ve yalnızca geliştiricinin kendi makinesinde, geliştirme sırasında kullanılır; `build` ise kodu tam tip kontrolünden (`tsc`) geçirip optimize ederek gerçek bir sunucuya yüklenebilecek statik `dist/` çıktısı üretir. İkincisi **kullanım yeri** farkıdır — geliştirme sunucusu üretimde (canlı, kullanıcıların eriştiği ortamda) asla kullanılmaz; yalnızca `build` çıktısı (`dist/` klasörü, örn. Hf.14'te nginx üzerinden) gerçek kullanıcılara sunulur.

Soru (Ünite 3 — `koltuk.durum = 'rezerve'` ataması). `tsc` bu satırı bir tip hatası olarak işaretler ve derleme/tip denetimi başarısız olur, çünkü `'rezerve'` metni,

`durum: 'bos' | 'secili' | 'dolu'` birleşim tipinde tanımlı üç değerden hiçbirine eşit değildir — TypeScript yalnızca bu üç tam değeri geçerli kabul eder. Bu davranış faydalıdır çünkü olası bir yazım hatasını (ör. “rezerve” yerine yanlışlıkla yazılmış bir durum adı) ya da unutulmuş bir durum geçişini kod hiç çalışmadan önce, editördeyken — kırmızı bir çizgiyle — yakalamanızı sağlar. Aynı kod düz JavaScript’te yazılsaydı, `durum` alanına `'rezerve'` atamak hiçbir uyarı vermeden sessizce geçerdi ve hata, belki de bir kullanıcının önünde, sayfa yanlış davrandığında ortaya çıkardı.

Soru (Ünite 4 — `setTimeout(fn, 0)` neden hemen çalışmaz). JavaScript tek iş parçacığında çalışır ve `setTimeout` süresi dolan işi doğrudan çalıştırmaz, onu bir görev kuyruğuna (makro görev kuyruğuna) ekler. Olay döngüsü, çağrı yığını tamamen boşalmadan — yani o an çalışan tüm senkron kod ve varsa bekleyen tüm mikro görevler (Promise callback’leri) bitmeden — kuyruktaki hiçbir işi almaz. `0` milisaniyelik gecikme yalnızca “mümkün olan en kısa sürede kuyruğa gir” anlamına gelir, “hemen çalış” anlamına gelmez; bu yüzden “0 ms” bile fiilen “sırada beklemek” demektir — mevcut senkron kod ne kadar uzun sürerse sürsün, `setTimeout` callback’i o kod bitmeden asla çalışmaz.